Objetivos da aula:

- Compreender a estrutura básica do NestJS (modules, controllers e services)
- Entender o papel dos decorators no funcionamento do framework
- Implementar um CRUD utilizando NestJS integrado ao Drizzle ORM
- Aplicar injeção de dependência na organização da aplicação
- Utilizar DTOs e ValidationPipe para validação de dados
- Implementar tratamento de erros com exceptions do NestJS

Repositório: https://github.com/arleysouza/tpII-2026-1-nestjs

Índice:

i. Fundamentos do NestJS e uso de decorators
ii. Implementação do CRUD com NestJS + Drizzle
iii. Tratamento de exceções, validação e robustez no NestJS

i. Fundamentos do NestJS e uso de decorators

1.1. Motivação: por que usar NestJS

Nas aulas anteriores, foram discutidos problemas comuns em aplicações backend:

- excesso de código em rotas
- mistura de responsabilidades
- dificuldade de manutenção
- alto acoplamento

Arquiteturas como MVC, MVP e MVVM surgem justamente para resolver esses problemas, propondo a separação entre:

- interface
- lógica
- dados

O NestJS surge como uma solução prática para aplicar esses princípios em aplicações Node.js.

Ele não apenas permite organizar o código — ele **obriga uma organização estruturada**.

No projeto da disciplina, essa organização já está presente:

- controller → entrada HTTP
- service → lógica
- database → acesso a dados
- schema → definição da estrutura do banco

1.2. O que é o NestJS (visão prática)

O NestJS é um framework backend baseado em Node.js e TypeScript que:

- utiliza Express (ou Fastify) internamente
- organiza a aplicação em módulos
- utiliza injeção de dependência
- faz uso intensivo de decorators

A principal diferença em relação a um projeto Express tradicional é:

no Nest, a estrutura da aplicação é explícita e padronizada.

1.3. Como o NestJS organiza uma aplicação

A aplicação é organizada em três elementos principais:

1. Controllers

Responsáveis por:

- receber requisições HTTP
- extrair dados
- retornar respostas

2. Services (Providers)

Responsáveis por:

- lógica de negócio
- acesso a dados
- regras da aplicação

3. Modules

Responsáveis por:

- agrupar funcionalidades
- organizar dependências

1.4. Fluxo de uma requisição no NestJS

Uma requisição percorre o seguinte caminho:

```
Requisição HTTP → Controller → Service → Banco → Service → Controller → Resposta
```

Esse fluxo é fundamental e deve ser compreendido desde o início.

1.5. Injeção de dependência (conceito central)

No NestJS, classes não criam suas dependências manualmente.

Exemplo tradicional (evitado):

```
const service = new UsersService();
```

No Nest:

```
constructor(private readonly usersService: UsersService) {}
```

O framework:

- cria a instância
- gerencia o ciclo de vida
- injeta automaticamente

Vantagens:

- baixo acoplamento
- melhor testabilidade
- maior organização

1.6. Decorators: a base do NestJS

O NestJS funciona a partir de decorators, que são anotações usadas para definir o papel de cada elemento.

Sem decorators, o Nest não consegue entender a aplicação.

Exemplo:

```
@Controller('api/users')
export class UsersController {}
```

Esse decorator informa que:

- a classe é um controller
- o prefixo da rota é `/api/users`

1.7. Tipos de decorators no NestJS

Os decorators podem ser classificados em três grupos principais:

1. Decorators de classe

Definem o papel da classe:

- @Module() → define um módulo
- @Controller() → define um controller
- @Injectable() → define um provider

2. Decorators de método

Definem endpoints HTTP:

```
@Get()
@Post()
@Put()
@Delete()
```

Exemplo:

```
@Get(':id')
findOne() {}
```

3. Decorators de parâmetro

Definem a origem dos dados:

```
@Body()
@Param()
```

Exemplo:

```
findOne(@Param('id', ParseIntPipe) id: number)
```

1.8. Módulos: organização da aplicação

Um módulo agrupa componentes relacionados.

Exemplo conceitual:

```
@Module({
  imports: [DatabaseModule],
  controllers: [UsersController],
  providers: [UsersService],
})
export class UsersModule {}
```

No projeto:

- AppModule → módulo raiz
- UsersModule → domínio de usuários
- DatabaseModule → acesso ao banco

1.9. Controllers: entrada da aplicação

Exemplo simplificado:

```
@Controller('api/users')
export class UsersController {
  constructor(private readonly usersService: UsersService) {}

  @Get()
  findAll() {
    return this.usersService.findAll();
  }
}
```

Responsabilidades:

- receber requisição
- chamar o service
- retornar resposta

1.10. Services: lógica da aplicação

Exemplo:

```
@Injectable()
export class UsersService {
  constructor(private readonly databaseService: DatabaseService) {}
}
```

Responsabilidades:

- implementar regras
- acessar banco
- tratar dados

1.11. DTOs e validação

DTO (Data Transfer Object) define o formato dos dados.

Exemplo:

```
export class UpdateUserDto {
  @IsString()
  @MaxLength(100)
  name?: string;
}
```

Funções:

- validar entrada
- documentar contrato da API
- evitar dados inválidos

1.12. ValidationPipe

Configurado no main.ts:

```
app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,
      transform: true,
      forbidNonWhitelisted: true,
    }),
);
```

Efeitos:

- remove campos inválidos
- converte tipos
- bloqueia dados inesperados

1.13. Pipes (exemplo prático)

```
@Param('id', ParseIntPipe) id: number
```

Função:

- transforma string → número
- valida entrada

Se inválido:

- erro automático
- requisição interrompida

1.14. Visão consolidada

A aplicação completa pode ser entendida assim:

- **Módulo** organiza
- **Controller** recebe
- **Service** processa
- **DatabaseService** acessa dados
- **DTO + Pipes** validam entrada

ii. Implementação do CRUD com NestJS + Drizzle

2.1. Introdução

Neste capítulo, o objetivo é compreender **como os conceitos do NestJS se materializam em código real**, utilizando o módulo de usuários do projeto.

Ao final, o você deve entender:

- como definir endpoints HTTP
- como estruturar um CRUD no Nest
- como integrar o Nest com o Drizzle ORM
- como os componentes interagem no fluxo completo

2.2. Estrutura do módulo de usuários

O CRUD está organizado dentro do `UsersModule`, que contém:

- `users.controller.ts` → define as rotas
- `users.service.ts` → implementa a lógica
- `users.schema.ts` → define a tabela (Drizzle)

Essa organização segue o padrão:

```
Controller → Service → Database
```

2.3. Visão geral do CRUD

As operações implementadas são:

- criar usuário (POST)
- listar usuários (GET)
- buscar por ID (GET)
- atualizar (PUT)
- remover (DELETE)

Cada operação segue o mesmo padrão:

```
HTTP → Controller → Service → Database → Service → Controller → HTTP
```

2.4. Controller: definição dos endpoints

O controller define as rotas da aplicação:

```
@Controller('api/users')
export class UsersController {
  constructor(private readonly usersService: UsersService) {}
}
```

Cada método do controller corresponde a um endpoint.

2.5. Create – inserção de dados

```
@Post()
create(@Body() createUserDto: CreateUserDto) {
  return this.usersService.create(createUserDto);
}
```

O que acontece:

1. requisição POST /api/users chega ao controller
2. corpo é mapeado para `CreateUserDto`
3. validação é aplicada automaticamente
4. service é chamado

2.6. Read – listagem de usuários

```
@Get()
findAll() {
  return this.usersService.findAll();
}
```

- rota: GET /api/users
- retorna todos os registros

2.7. Read – busca por ID

```
@Get(':id')
findOne(@Param('id', ParseIntPipe) id: number) {
  return this.usersService.findOne(id);
}
```

Pontos importantes:

- `:id` → parâmetro de rota
- `ParseIntPipe` → valida e converte

2.8. Update – atualização

```
@Put(':id')
update(
  @Param('id', ParseIntPipe) id: number,
  @Body() updateUserDto: UpdateUserDto,
) {
  return this.usersService.update(id, updateUserDto);
}
```

- combina parâmetro de rota + corpo da requisição
- permite atualização parcial

2.9. Delete – remoção

```
@Delete(':id')
remove(@Param('id', ParseIntPipe) id: number) {
  return this.usersService.remove(id);
}
```

- remove o registro pelo ID

2.10. Service: onde a lógica acontece

O `UsersService` implementa o comportamento do sistema:

```
@Injectable()
export class UsersService {
  constructor(private readonly databaseService: DatabaseService) {}
}
```

O service:

- não conhece HTTP
- não define rotas
- concentra a lógica do CRUD

2.11. Inserção com Drizzle

```
async create(createUserDto: CreateUserDto) {
  const [user] = await this.databaseService.db
    .insert(users)
    .values({
      name: createUserDto.name,
      email: createUserDto.email ?? null,
    })
    .returning();

  return user;
}
```

Observações:

- `.insert()` → operação SQL
- `.values()` → dados
- `.returning()` → retorna resultado

2.12. Consulta com Drizzle

```
findAll() {
  return this.databaseService.db
    .select()
    .from(users)
    .orderBy(asc(users.idUser));
}
```

- retorna lista ordenada
- tipagem automática

2.13. Consulta por ID

```
async findOne(id: number) {
  const [user] = await this.databaseService.db
    .select()
    .from(users)
    .where(eq(users.idUser, id))
    .limit(1);

  if (!user) {
    throw new NotFoundException(`Usuário ${id} não encontrado.`);
  }

  return user;
}
```

- Drizzle retorna array

- necessário acessar o primeiro elemento

2.14. Atualização

```
async update(id: number, updateUserDto: UpdateUserDto) {
  const existingUser = await this.findOne(id);

  const [user] = await this.databaseService.db
    .update(users)
    .set({
      name: updateUserDto.name ?? existingUser.name,
      email:
        updateUserDto.email !== undefined
          ? updateUserDto.email || null
          : existingUser.email,
    })
    .where(eq(users.idUser, id))
    .returning();

  return user;
}
```

2.15. Remoção

```
async remove(id: number) {
  const deletedUsers = await this.databaseService.db
    .delete(users)
    .where(eq(users.idUser, id))
    .returning({ idUser: users.idUser });

  if (!deletedUsers.length) {
    throw new NotFoundException(`Usuário ${id} não encontrado.`);
  }

  return {
    message: 'Usuário removido com sucesso.',
  };
}
```

2.16. Papel dos DTOs no CRUD

Os DTOs definem o contrato da API.

CreateUserDto

- campos obrigatórios
- validação de entrada

UpdateUserDto

- campos opcionais

- permite atualização parcial

2.17. Integração com DatabaseService

O acesso ao banco ocorre via:

```
this.databaseService.db
```

Isso garante:

- reutilização da conexão
- desacoplamento
- centralização da infraestrutura

2.18. Separação de responsabilidades (visão final)

No CRUD implementado:

- **Controller** → recebe HTTP
- **DTO/Pipes** → validam dados
- **Service** → aplica lógica
- **DatabaseService** → fornece acesso ao banco
- **Drizzle** → executa SQL

iii. Tratamento de exceções, validação e robustez no NestJS

3.1. Introdução

Após implementar o CRUD, surge um novo problema:

como garantir que a aplicação se comporte corretamente em situações de erro?

Neste capítulo, o aluno deve compreender:

- como o NestJS trata erros HTTP
- como validar dados de forma robusta
- como evitar comportamentos inconsistentes
- como aplicar boas práticas no backend

3.2. Problemas em aplicações sem tratamento adequado

Sem tratamento estruturado, erros costumam ser tratados assim:

```
if (!user) {
  throw new Error("Usuário não encontrado");
}
```

Problemas dessa abordagem:

- não define status HTTP
- resposta inconsistente
- difícil padronização
- mistura lógica com controle de erro

3.3. Exceptions no NestJS

O NestJS fornece exceções prontas que já representam erros HTTP.

Exemplo:

```
throw new NotFoundException(`Usuário ${id} não encontrado.`);
```

Essa exception gera automaticamente:

- status HTTP 404
- resposta padronizada
- mensagem clara

3.4. Principais exceptions utilizadas

Exception	Código HTTP	Uso
BadRequestException	400	Dados inválidos
NotFoundException	404	Recurso não encontrado
ConflictException	409	Conflito (ex.: duplicidade)
InternalServerErrorException	500	Erro interno

3.5. Aplicando exceptions no service

Exemplo: busca por ID

```
async findOne(id: number) {
  const [user] = await this.databaseService.db
    .select()
    .from(users)
    .where(eq(users.idUser, id))
    .limit(1);

  if (!user) {
    throw new NotFoundException(`Usuário ${id} não encontrado.`);
  }

  return user;
}
```

Resultado:

- elimina retornos indefinidos
- torna o erro explícito
- melhora a API

3.6. Validação como primeira linha de defesa

Antes mesmo de chegar ao service, os dados já podem ser validados.

Isso é feito com:

- DTOs
- decorators do class-validator
- ValidationPipe

3.7. DTO com validação

Exemplo: CreateUserDto

```
export class CreateUserDto {
  @IsString()
  @IsNotEmpty()
  @MinLength(3, { message: 'O nome deve ter pelo menos 3 caracteres.' })
  @MaxLength(100)
  name!: string;

  @IsOptional()
  @ValidateIf((_, value) => value !== null && value !== '')
  @IsEmail({}, { message: 'O e-mail informado deve ter um formato válido.' })
  @MaxLength(100)
  email?: string | null;
}
```

O que isso garante:

- tipo correto
- valor obrigatório
- formato válido

3.8. ValidationPipe (execução automática)

```
app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,
      transform: true,
      forbidNonWhitelisted: true,
    }),
);
```

Efeitos:

- remove propriedades não permitidas
- transforma tipos automaticamente
- bloqueia dados inesperados

3.9. Pipes como validação adicional

Exemplo no controller:

```
@Param("id", ParseIntPipe) id: number
```

Função:

- converte string → número
- valida automaticamente

Se inválido:

- erro 400
- execução interrompida

3.10. Fluxo com validação e erro

Agora o fluxo completo passa a ser:

```
Request → DTO/Pipes → Controller → Service → Banco → Response
```

Se houver erro:

```
Erro → Exception → Response HTTP padronizada
```

Exercícios

Exercício 1 – Altere o código de exemplo para que o campo `name` da tabela `users` seja único (`UNIQUE`).

Além disso, ao inserir ou atualizar um registro, sua aplicação deverá:

- verificar se já existe um usuário com o mesmo nome
- lançar a exceção `ConflictException` caso haja duplicidade

A estrutura da tabela deve ser a seguinte:

```
CREATE TABLE users (
  id_user SERIAL NOT NULL,
  name VARCHAR(100) NOT NULL UNIQUE,
  email VARCHAR(100),
  PRIMARY KEY(id_user)
);
```

Exercício 2 – Altere o código do projeto para incluir a tabela `cars`, implementando o relacionamento de chave estrangeira com a tabela `users`.

Além disso, mantenha a organização do projeto seguindo a estrutura do NestJS, separando adequadamente:

- controller
- service
- schema

A estrutura da tabela deve ser:

```
CREATE TABLE cars (
  id_car SERIAL NOT NULL,
  id_user INTEGER NOT NULL,
  plate CHAR(7) NOT NULL,
  PRIMARY KEY(id_car),
  FOREIGN KEY(id_user)
    REFERENCES users(id_user)
);
```

Requisitos adicionais

- A placa (plate) deve ser validada no padrão Mercosul (padrão: AAA1A11)
- A validação deve ser implementada utilizando DTOs e class-validator
- Caso a placa informada seja inválida, a aplicação deve retornar erro (`BadRequestException`)